

建立第一個 Keras 模型 (採用遷移學習)

來源：<https://codelabs.developers.google.com/codelabs/keras-flowers-transfer-learning?hl=zh-tw>

步驟數：5

在本研究室中，您將瞭解如何建構 Keras 分類器。我們不會嘗試找出最佳的類神經網路層組合來識別花朵，而是先使用稱為遷移學習的技術，將強大的預先訓練模型調整至我們的資料集。本研究室包含有關類神經網路的必要理論解釋，也是開發人員學習深度學習的好起點。

目錄

1. [1. 總覽](#)
2. [2. Google Colaboratory 快速入門](#)
3. [3. \[INFO\] Neural network classifier 101](#)
4. [4. 遷移學習](#)
5. [5. 恭喜！](#)

步驟 1 · 約 1 分鐘

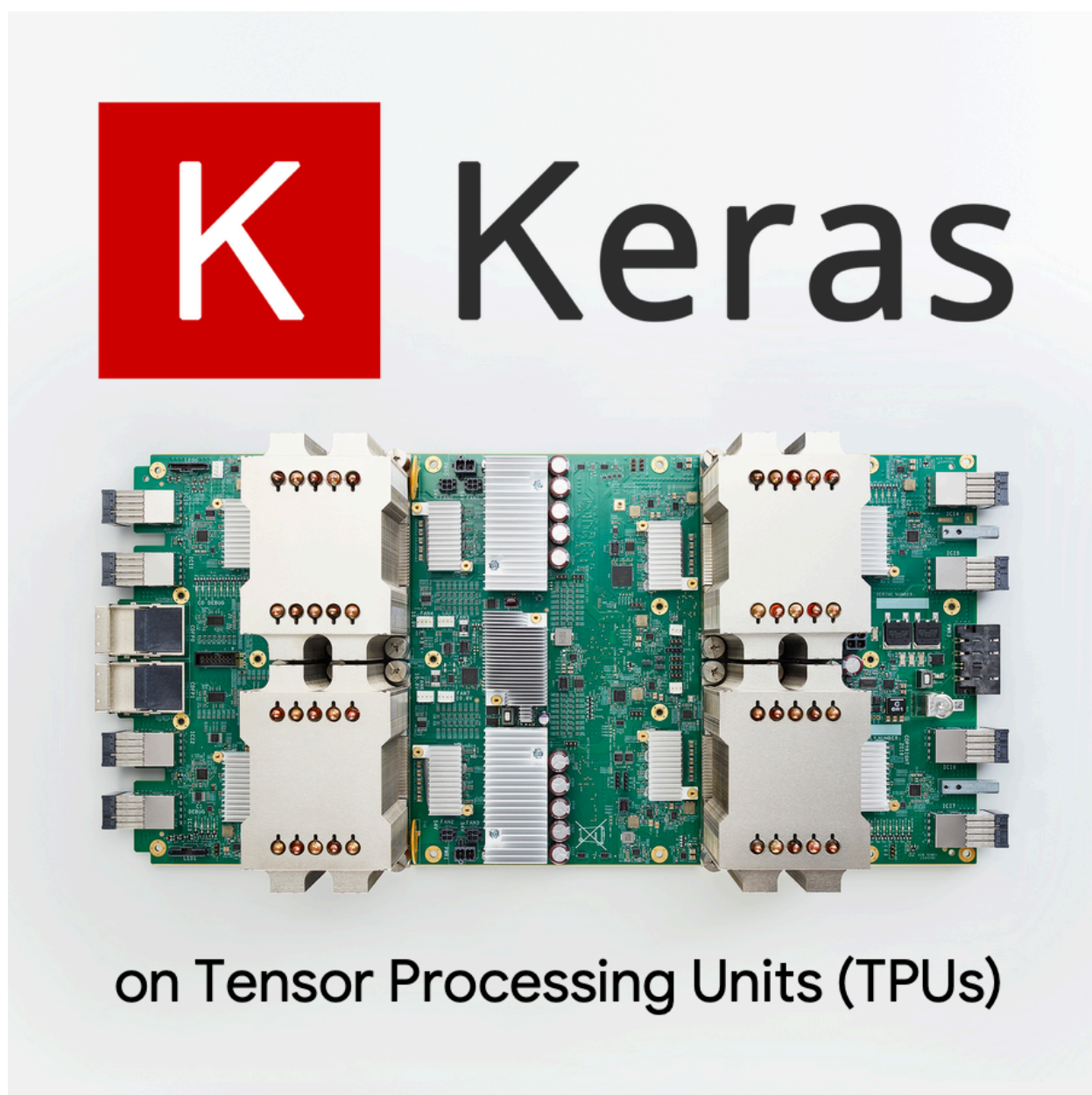
1. 總覽

在本實驗室中，您將學習如何建構 Keras 分類器。我們不會嘗試找出可辨識花朵的完美類神經網路層組合，而是先使用遷移學習技術，讓強大的預先訓練模型適應我們的資料集。

本實驗室包含類神經網路的必要理論說明，是開發人員學習深度學習的良好起點。

本實驗室是「在 TPU 上使用 Keras」系列實驗室的第 2 部分。你可以依下列順序執行這些步驟，也可以單獨執行。

- [TPU 高速資料管道：tf.data.Dataset 和 TFRecords](#)
- [\[本實驗室\] 建立第一個 Keras 模型 \(採用遷移學習\)](#)
- [卷積類神經網路 \(搭配使用 Keras 和 TPU\)](#)
- [搭配使用 Keras 和 TPU，翻新 convnets、squeezeNet 和 Xception 模型](#)



課程內容

- 如要使用 softmax 層和交叉熵損失，建構自己的 Keras 圖片分類器
- 如要作弊 🐱，請使用遷移學習，而不是自行建構模型。

意見回饋

如果您發現這個程式碼研究室有任何錯誤，請告訴我們。你可以透過 GitHub 問題 [\[意見回饋連結\]](#) 提供意見。

步驟 2 · 約 2 分鐘

2. Google Colaboratory 快速入門

本實驗室使用 Google Colaboratory，您無須進行任何設定。Colaboratory 是線上筆記本平台，適用於教育用途。提供免費的 CPU、GPU 和 TPU 訓練。

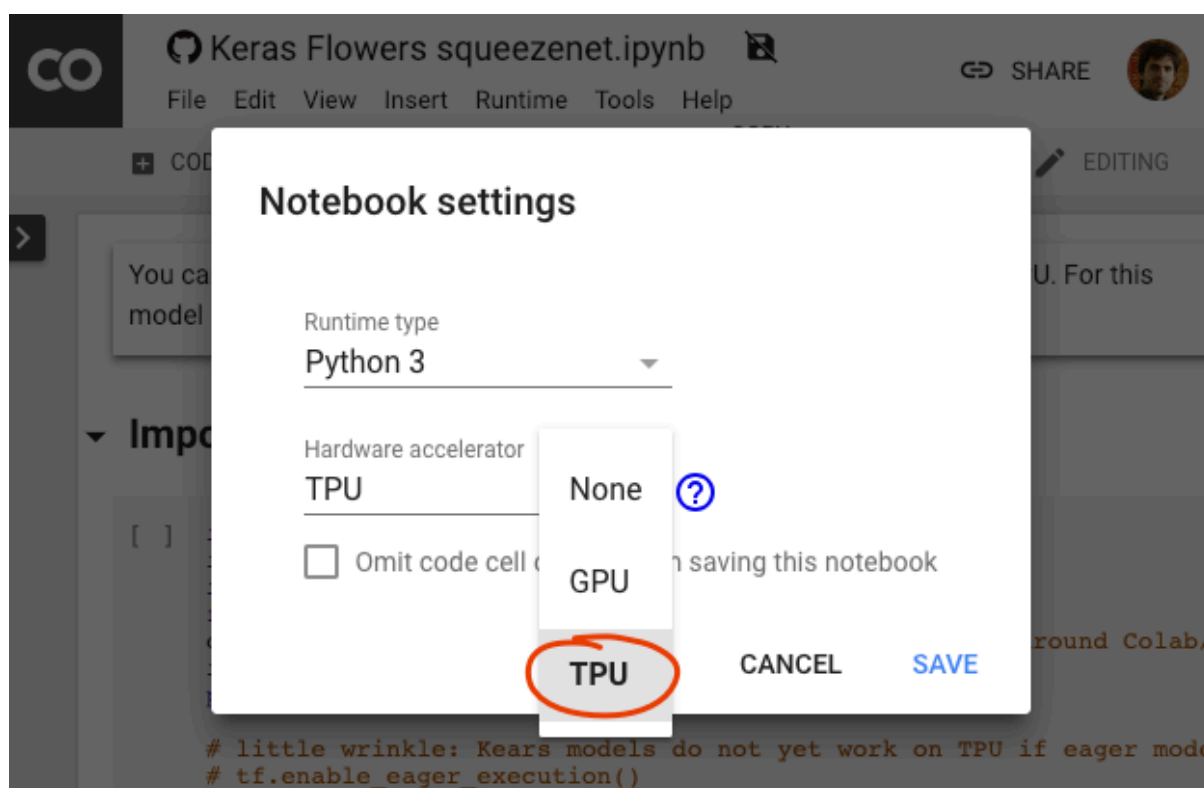
If this is old news to you **SKIP** ✕ to the next exercise otherwise **READ ON** ↘

您可以開啟這個範例筆記本，並執行幾個儲存格，熟悉 Colaboratory 的操作方式。



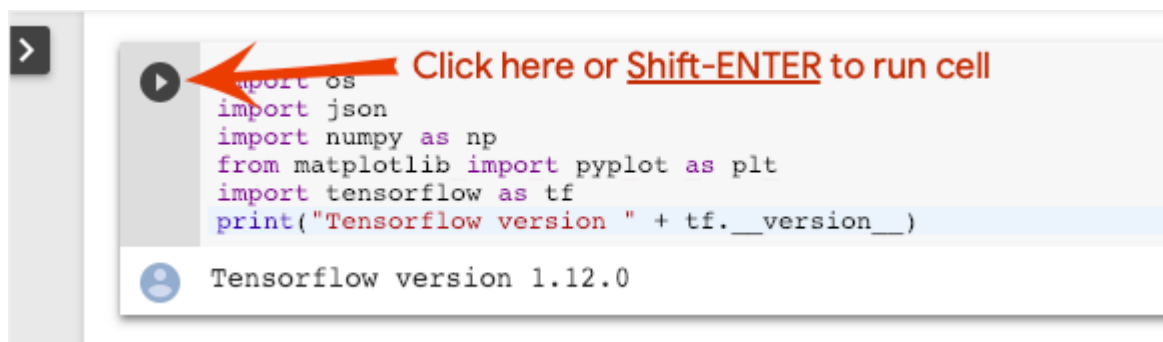
Welcome to Colab.ipynb

選取 TPU 後端



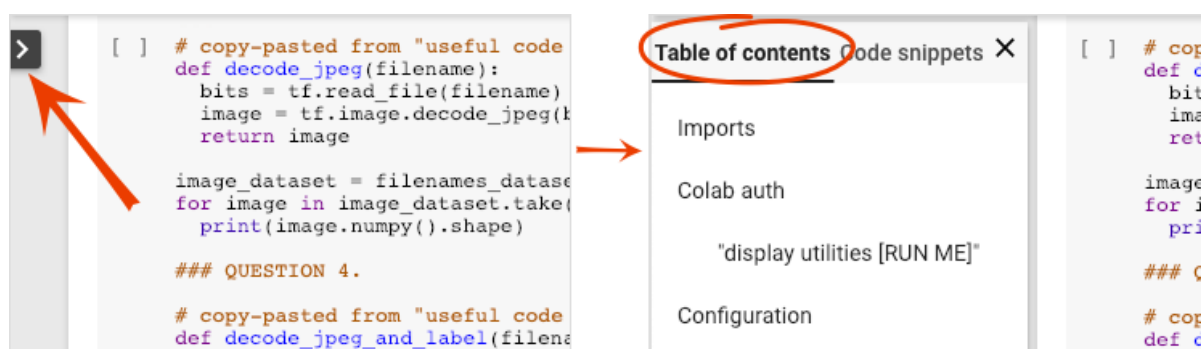
在 Colab 選單中，依序選取「執行階段」>「變更執行階段類型」，然後選取「TPU」。在本程式碼研究室中，您將使用強大的 TPU (Tensor Processing Unit)，支援硬體加速訓練。首次執行時，系統會自動連線至執行階段，您也可以使用右上角的「連線」按鈕。

筆記本執行



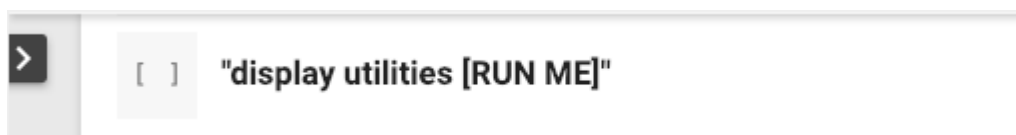
按一下儲存格並使用 Shift-ENTER，即可一次執行一個儲存格。您也可以依序選取「執行階段」>「全部執行」，執行整個筆記本。

Table of contents



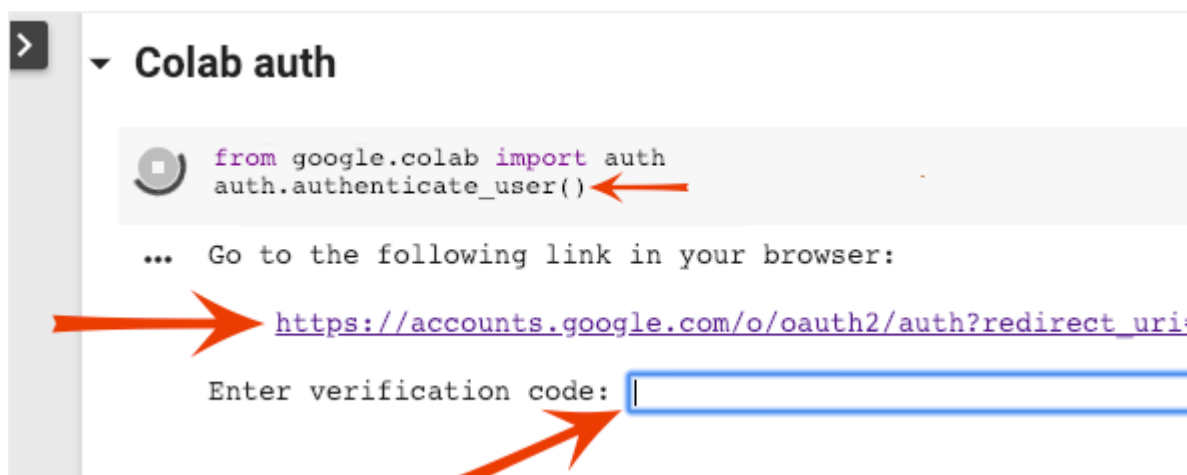
所有筆記本都有目錄。你可以使用左側的黑色箭頭開啟這個選單。

隱藏的儲存格



部分儲存格只會顯示標題，這是 Colab 專屬的筆記本功能。您可以按兩下查看其中的程式碼，但通常不會很有趣。通常是支援或視覺化函式。您仍須執行這些儲存格，才能定義其中的函式。

驗證



只要使用已授權的帳戶進行驗證，Colab 就能存取您的私人 Google Cloud Storage bucket。上述程式碼片段會觸發驗證程序。

3. [INFO] Neural network classifier 101

簡介

如果下一段**粗體**字詞你都瞭解，即可直接進行下一個練習。如果你才剛開始接觸深度學習，歡迎閱讀下文。

類神經網路分類器是由多個**神經元層**組成。如果是圖片分類，這些可以是**密集**或更常見的**捲積層**。通常會使用 **relu** 活化函數啟動。最後一層使用的神經元數量與類別數量相同，並以 **softmax** 啟動。對於分類，**交叉熵**是最常用的損失函式，可比較 **one-hot** 編碼的**標籤** (即正確答案) 與類神經網路預測的機率。為盡量減少損失，最好選擇具有動量的最佳化工具，例如 **AdamOptimizer**，並在訓練圖片和標籤的**批次**上進行訓練。

如要以層序列建構模型，Keras 提供 Sequential API。舉例來說，使用三個密集層的圖片分類器可以 Keras 撰寫，如下所示：

```
model = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=[192, 192, 3]),
    tf.keras.layers.Dense(500, activation="relu"),
    tf.keras.layers.Dense(50, activation="relu"),
    tf.keras.layers.Dense(5, activation='softmax') # classifying into 5 classes
])

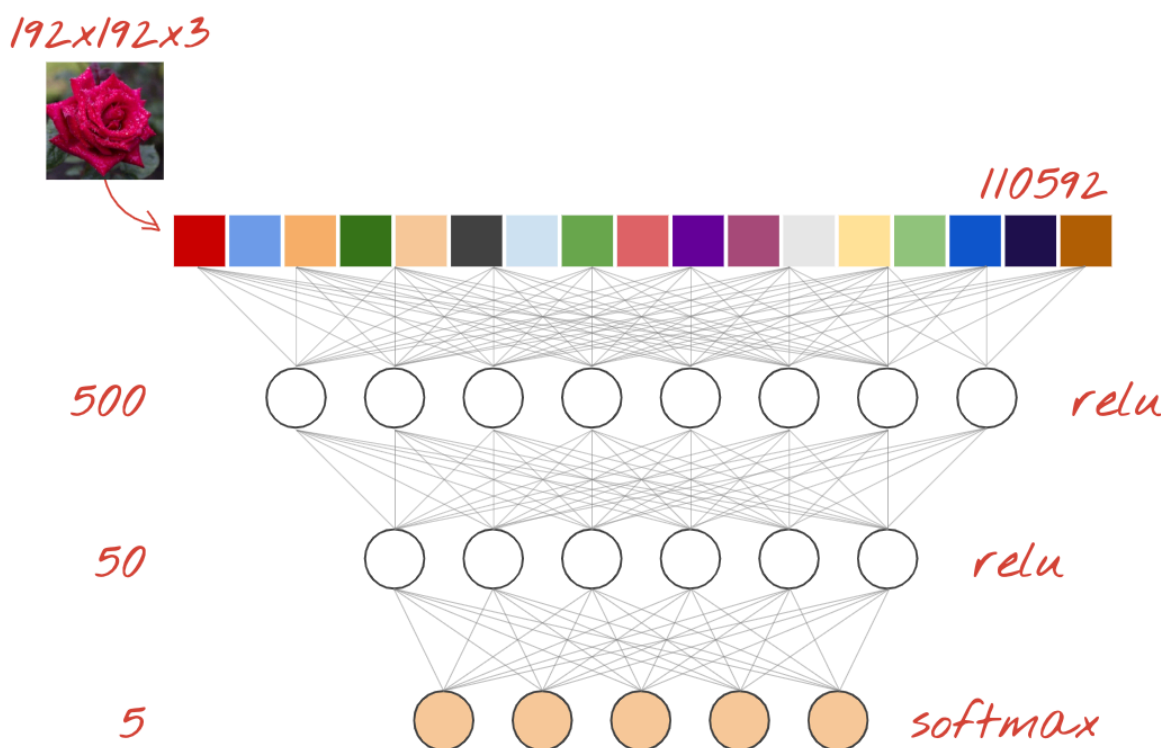
# this configures the training of the model. Keras calls it "compiling" the model.
model.compile(
    optimizer='adam',
    loss= 'categorical_crossentropy',
    metrics=['accuracy']) # % of correct answers

# train the model
model.fit(dataset, ... )
```

If this is old news to you **SKIP**  to the next exercise otherwise **READ ON** 

密集類神經網路

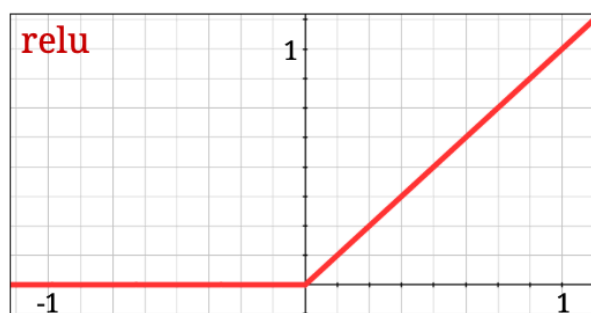
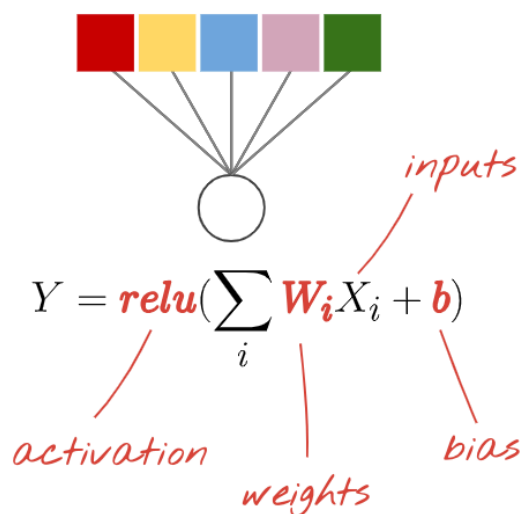
這是最簡單的圖片分類類神經網路，是由分層排列的「神經元」組成。第一層會處理輸入資料，並將輸出內容饋送至其他層。由於每個神經元都會連結至上一層的所有神經元，因此稱為「密集」。



您可以將圖像的所有像素 RGB 值攤平成長向量，並做為輸入內容，饋送至這類網路。這並非圖像辨識的最佳技術，但我們之後會加以改良。

神經元、啟動、RELU

「神經元」會計算所有輸入的加權總和，加上稱為「偏差」的值，然後透過所謂的「活化函數」饋送結果。權重和偏差一開始是未知的。這些權重會隨機初始化，並透過大量已知資料訓練類神經網路「學習」。



最常用的活化函數稱為 RELU，也就是線性整流函數。如上圖所示，這項函式非常簡單。

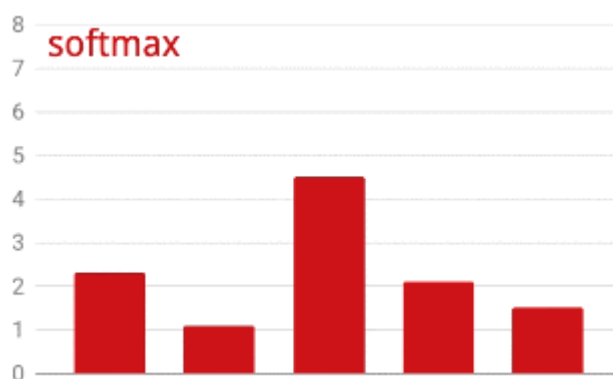
Softmax 啟用

上述網路以 5 個神經元的層級結尾，因為我們要將花朵分類為 5 個類別 (玫瑰、鬱金香、蒲公英、雛菊、向日葵)。中間層的神經元會使用經典的 ReLU 活化函數啟動。不過，在最後一層中，我們想計算介於 0 和 1 之間的數字，代表這朵花是玫瑰、鬱金香等的機率。為此，我們會使用名為「softmax」的活化函數。

對向量套用 Softmax 時，會先計算每個元素的指數，然後將向量正規化，通常是使用 L1 範數 (絕對值總和)，使值加總為 1，並可解讀為機率。

$$\text{softmax}(L_n) = \frac{e^{L_n}}{\|e^L\|}$$

weighted sum+bias *L1 norm*



為什麼「softmax」稱為 softmax？指數是急遽遞增的函式。這會增加神經元輸出之間的差異。接著，當您將向量正規化時，主導常態的最大元素會正規化為接近 1 的值，而所有其他元素則會除以較大的值，並正規化為接近 0 的值。產生的向量會清楚顯示哪個是勝出類別 (即「max」)，但會保留值的原始相對順序，因此是「soft」。

交叉熵損失

現在類神經網路會根據輸入圖片產生預測結果，我們需要測量預測結果的準確度，也就是網路提供的結果與正確答案 (通常稱為「標籤」) 之間的距離。請注意，資料集中的所有圖片都有正確標籤。

任何距離都適用，但對於分類問題，所謂的「交叉熵距離」**最有效**。我們將此稱為誤差或「損失」函式：

1	2	3	4	5
0	0	1	0	0

actual probabilities,
"one-hot" encoded

Cross entropy: $-\sum Y_i' \cdot \log(Y_i)$

computed probabilities

0.2	0.1	0.1	0.5	0.1
-----	-----	-----	-----	-----

1 2 3 4 5
predicted class

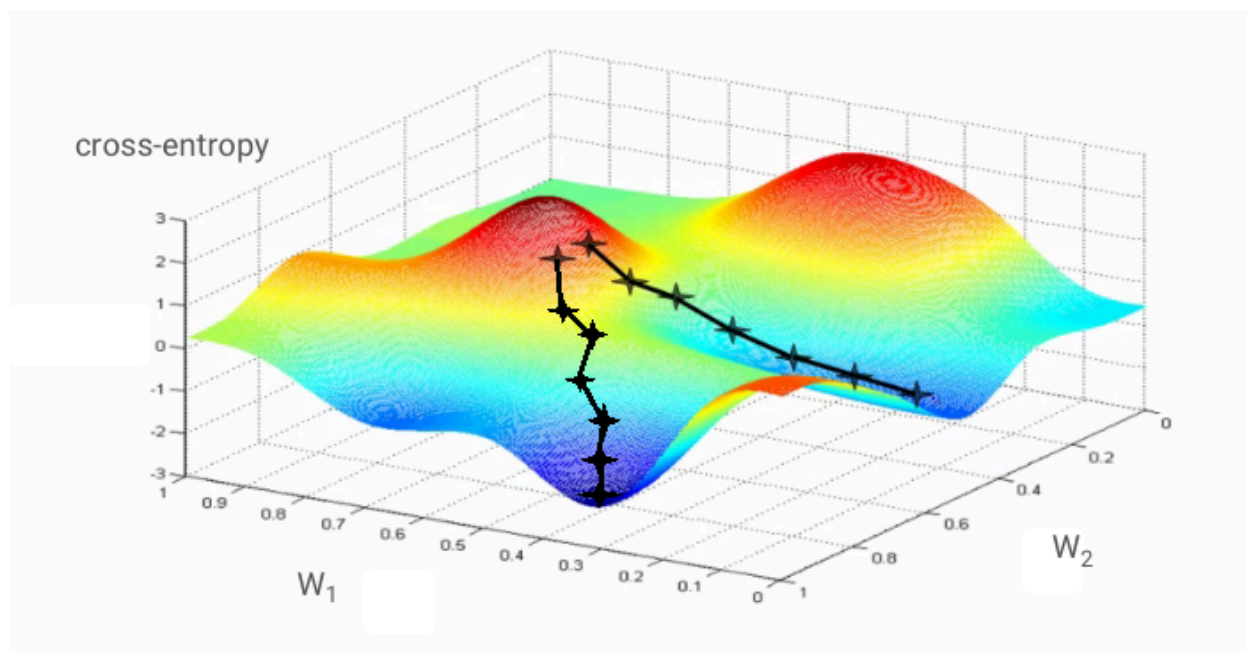
「One-hot」編碼是指使用 5 個值的向量表示「3) 蒲公英」標籤，除了第 3 個值為 1 以外，其餘值皆為零。這個向量代表是蒲公英的機率為 100%。我們的類神經網路也會以 5 個機率值的向量輸出預測結果。方便比較。

梯度下降法

「訓練」類神經網路實際上是指使用訓練圖片和標籤調整權重和偏誤，盡量減少交叉熵損失函式。運作方式如下：

交叉熵是權重、偏差、訓練圖片的像素及其已知類別的函式。

如果我們計算相對於所有權重和所有偏差的交叉熵偏導數，就會得到「梯度」，這是針對特定圖片、標籤，以及權重和偏差的現值計算而得。請注意，我們可能有數百萬個權重和偏差，因此計算梯度聽起來像是大量的工作。幸好，Tensorflow 會為我們執行這項作業。梯度的數學特性是「向上」指向。由於我們希望交叉熵較低，因此要朝相反方向移動。我們會根據梯度的一小部分更新權重和偏差值。接著，我們會在訓練迴圈中，使用下一批訓練圖片和標籤，重複執行相同的操作。希望這會收斂到交叉熵最小的位置，但無法保證這個最小值是唯一的。



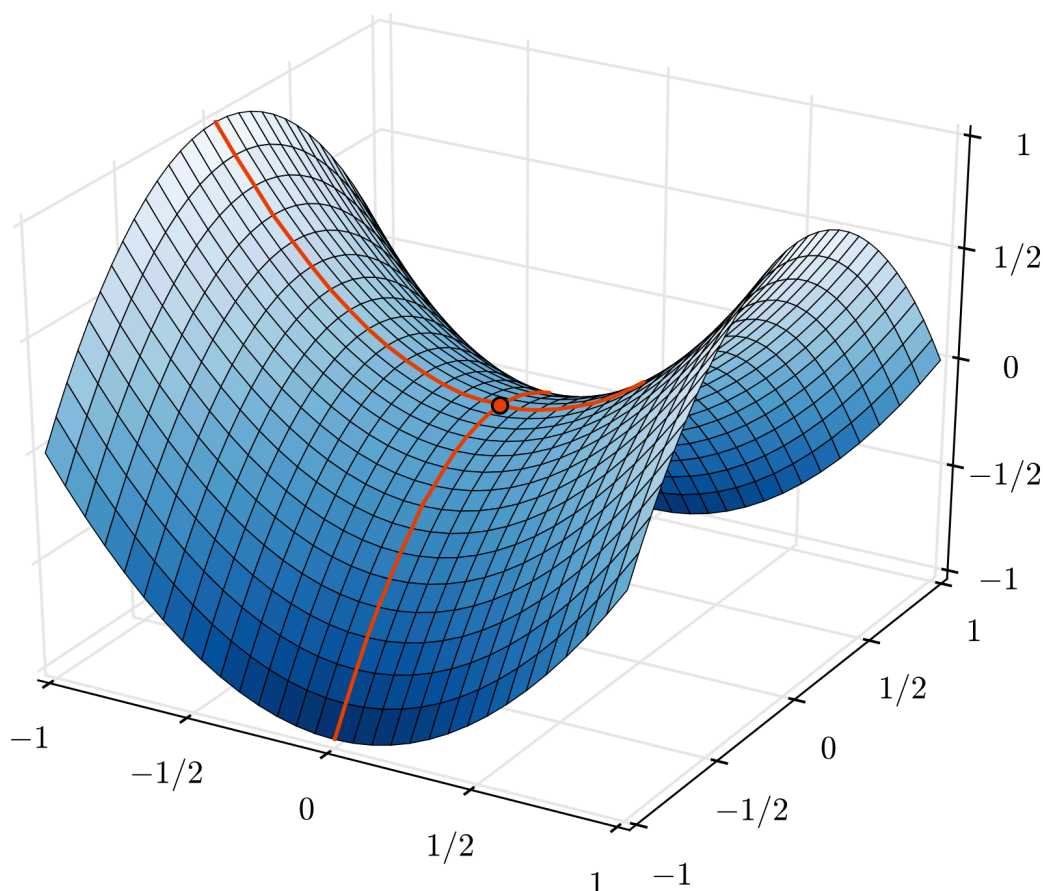
「學習率」：您無法在每次疊代中，依據整個梯度長度更新權重和偏差值。這就像穿著七里靴試圖抵達谷底，您會從山谷的一側跳到另一側。如要到達底部，您必須採取較小的步，也就是只使用一小部分的梯度，通常在 $1/1000$ 範圍內。這個分數稱為「學習率」。

小批次和動量

您可以在單一範例圖片上計算梯度，並立即更新權重和偏差，但如果對一批 (例如 128 張) 圖片執行這項操作，梯度就能更準確地呈現不同範例圖片施加的限制，因此更有可能更快收斂至解決方案。小批次的大小是可調整的參數。

這項技術有時稱為「隨機梯度下降」，還有另一項更實用的優點：處理批次資料也代表要處理較大的矩陣，而這類矩陣通常更容易在 GPU 和 TPU 上進行最佳化。

不過，收斂過程可能還是有點混亂，如果梯度向量全為零，甚至可能會停止。這是否表示我們已找到最小值？不一定。梯度分量在最小值或最大值時可為零。如果梯度向量有數百萬個元素，且全為零，則每個零都對應到最小值，且沒有任何一個對應到最大值的機率相當小。在多維空間中，鞍點相當常見，我們不希望停留在鞍點。



插圖：鞍點。梯度為 0，但並非所有方向的最小值。(圖片出處：[Wikimedia](#)：作者 [Nicoguaro](#) - Own work，CC BY 3.0)

解決方法是為最佳化演算法增加一些動量，讓演算法能順利通過鞍點，不會停滯不前。

Tensorflow 程式庫提供各種最佳化工具，從 `tf.train.GradientDescentOptimizer` 開始。內建動能的進階熱門最佳化工具為 `tf.train.RMSPropOptimizer` 或 `tf.train.AdamOptimizer`。

詞彙

批次或小批：訓練一律會對訓練資料和標籤批次執行。這麼做有助於演算法收斂。「批次」維度通常是資料張量的第一個維度。舉例來說，形狀為 `[100, 192, 192, 3]` 的張量包含 100 張 192x192 像素的圖片，每個像素有三個值 (RGB)。

交叉熵損失：分類器常用的特殊損失函式。

稠密層：神經元層，其中每個神經元都會連結至前一層的所有神經元。

特徵：神經網路的輸入內容有時稱為「特徵」。找出要將資料集中的哪些部分 (或部分組合) 輸入類神經網路，才能獲得準確預測結果，這門技術稱為「特徵工程」。

標籤：監督式分類問題中的「類別」或正確答案的別名

學習率：在訓練迴圈的每次疊代中，權重和偏差值更新的梯度比例。

logits：在套用活化函數之前，神經元層的輸出內容稱為「logits」。這個詞彙源自「邏輯函數」，又稱「S 函數」，這是最常用的活化函數。「Neuron outputs before logistic function」縮短為「logits」。

損失：比較類神經網路輸出內容與正確答案的錯誤函式

神經元：計算輸入內容的加權總和、加入偏差，並透過活化函數提供結果。

One-hot 編碼：5 個類別中的第 3 個類別會編碼為 5 個元素的向量，除了第 3 個元素是 1 以外，其餘都是零。

relu：線性整流函數。神經元的熱門活化函數。

sigmoid：另一種曾廣受歡迎的活化函數，在特殊情況下仍有用處。

softmax：一種特殊活化函數，可作用於向量、增加最大元件與其他元件之間的差異，並將向量正規化為總和為 1，因此可解讀為機率向量。用做分類器的最後一個步驟。

張量：「張量」類似矩陣，但維度數量任意。1 維張量是向量。2 維張量是矩陣。然後，您就可以使用 3、4、5 個以上的維度張量。

4. 遷移學習

如果是圖像分類問題，密集層可能不夠。我們必須瞭解捲積層，以及排列這些層的許多方式。

但我們也可以走捷徑！您可以下載經過完整訓練的卷積類神經網路。您可以將最後一層 (softmax 分類標頭) 截斷，並換成自己的標頭。所有訓練權重和偏差都會維持不變，您只會重新訓練新增的 Softmax 層。這項技術稱為「遷移學習」，只要預先訓練類神經網路的資料集與您的資料集「夠接近」，這項技術就能發揮作用。

實作

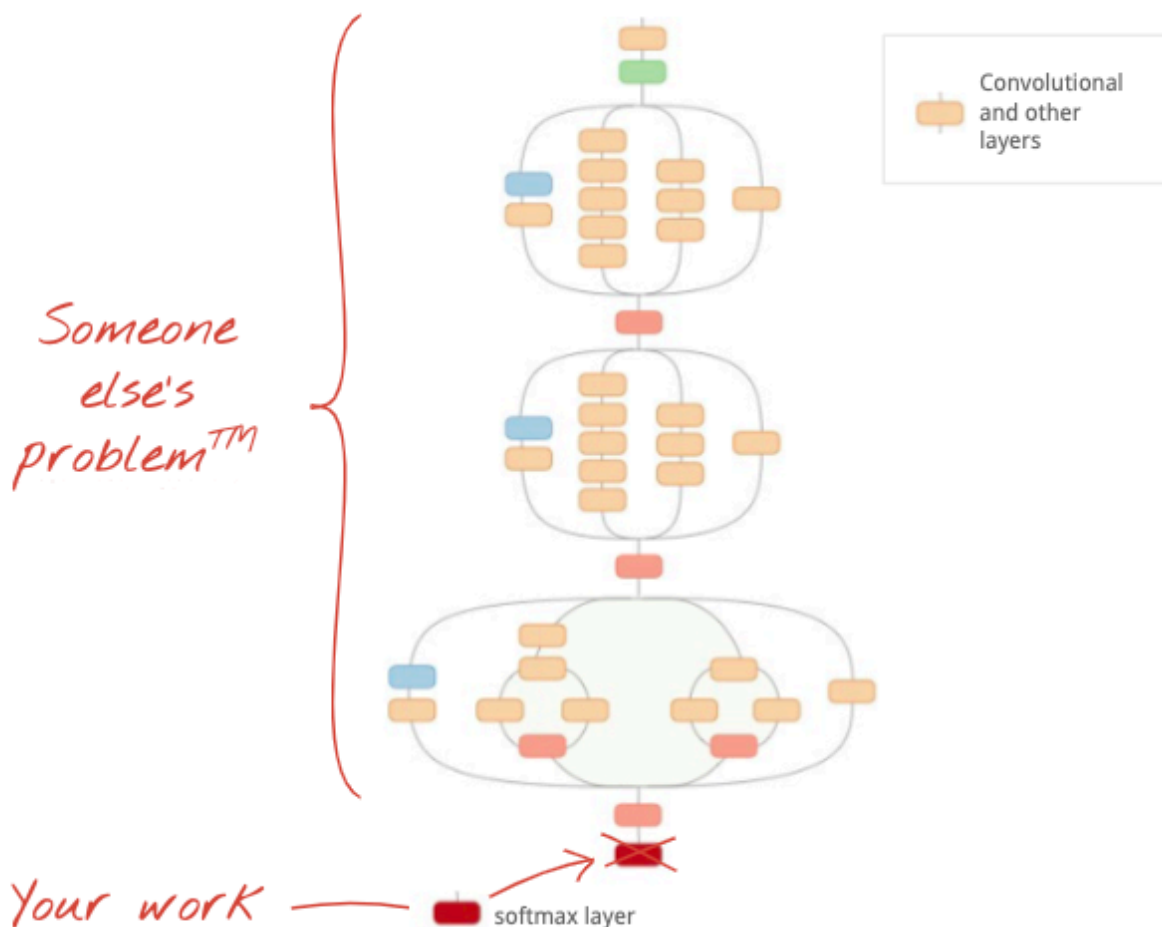
請開啟下列筆記本、執行儲存格 (Shift-ENTER)，並按照標示「WORK REQUIRED」的指示操作。



[Keras Flowers transfer learning \(playground\).ipynb](#)

其他資訊

透過遷移學習，您可同時享有頂尖研究人員開發的先進卷積類神經網路架構，以及在龐大圖片資料集上預先訓練的優勢。在本例中，我們將從 ImageNet 訓練的網路進行遷移學習。ImageNet 是包含許多植物和戶外場景的圖片資料庫，與花卉的相似度相當高。



插圖：使用已訓練的複雜卷積類神經網路做為黑箱，僅重新訓練分類標頭。這就是遷移學習。我們稍後會瞭解這些複雜的捲積層排列方式如何運作。目前這是其他人的問題。

Keras 中的遷移學習

在 Keras 中，您可以從 `tf.keras.applications.*` 集合例項化預先訓練模型。舉例來說，MobileNet V2 是非常出色的卷積架構，大小合理。選取 `include_top=False` 即可取得預先訓練模型，但沒有最終的 Softmax 層，因此您可以自行新增：

```
pretrained_model = tf.keras.applications.MobileNetV2(input_shape=[*IMAGE_SIZE, 3],
include_top=False)
pretrained_model.trainable = False

model = tf.keras.Sequential([
    pretrained_model,
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(5, activation='softmax')
])
```

另請注意 `pretrained_model.trainable = False` 設定。這會凍結預先訓練模型的權重和偏誤，因此您只會訓練 Softmax 層。這通常只涉及相對較少的權重，而且可以快速完成，不需要非常龐大的資料集。不過，如果您有大量資料，遷移學習搭配 `pretrained_model.trainable = True` 的效果會更好。預先訓練的權重可提供絕佳的初始值，且仍可透過訓練調整，以更符合您的問題。

最後，請注意插入密集 softmax 層之前的 `Flatten()` 層。密集層可處理資料的平坦向量，但我們不知道預先訓練的模型是否會傳回這類向量。因此我們需要將其攤平。在下一章深入探討卷積架構時，我們會說明卷積層傳回的資料格式。

使用這種方法，準確率應可接近 75%。

解決方案

解決方案筆記本在此。如果遇到困難，可以參考這項工具。



[Keras Flowers transfer learning \(solution\).ipynb](#)

涵蓋內容

- 🤖 如何在 Keras 中編寫分類器
- 🧠，並設定 softmax 最後一層和交叉熵損失
- 🐙 遷移學習
- 🧑‍🔬 訓練第一個模型
- 📊 追蹤訓練期間的損失和準確率

請花點時間在腦中瀏覽這份檢查清單。

步驟 5 · 約 0 分鐘

5. 恭喜！

您現在可以建構 Keras 模型。請繼續下一個實驗室，瞭解如何組裝捲積層。

- [TPU 高速資料管道](#)：tf.data.Dataset 和 TFRecords
- [\[本實驗室\] 建立第一個 Keras 模型 \(採用遷移學習\)](#)
- [卷積類神經網路 \(搭配使用 Keras 和 TPU\)](#)
- [搭配使用 Keras 和 TPU，翻新 convnets、squeezeNet 和 Xception 模型](#)

TPU 實務

TPU 和 GPU 可在 [Cloud AI Platform](#) 上使用：

- 在 [Deep Learning VM](#) 上
- 在 [AI 平台筆記本](#) 中
- 在 [AI Platform Training](#) 工作中

最後，我們非常重視意見回饋。如果您在本實驗室中發現任何錯誤，或認為有需要改進之處，請告訴我們。你可以透過 GitHub 問題 [\[意見回饋連結\]](#) 提供意見。



作者：Martin Görner
Twitter：[@martin_gorner](#)



www.tensorflow.org